Designing a Universal Information Language

Oscar Bender-Stone

March 26, 2026

DRAFT

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

Introduction

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3]Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.

DRAFT

[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Inquiry: what *is* information?

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Inquiry: what *is* information?
  - Depends on who you ask!

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.

DRAFT

[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Inquiry: what *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.

DRAFT

[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Inquiry: what *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.

DRAFT

[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Inquiry: what *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.

DRAFT

[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Introduction

- Information helps us **store** and **organize** data
  - Throughout *all* history: so much data!
  - Want to actually use in, e.g., decisions
- Inquiry: what *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]
    - Can be a *process*, or *knowledge*
    - Most useful as *thing*
  - **Floridi:** Information is **truthful**[3]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Challenges in Managing Information

DRAFT

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
- **Breadth:** wide variety of domains

DRAFT

Challenges in Managing Information

- **Depth:** domain specific data
  - Terminology, kinds of data, etc.
- **Breadth:** wide variety of domains
  - Different principles, methods
  - Example: Physics $\neq$ Literature!
- **Standardization:** ambiguous interpretations
  - Related issue: being **interoperable**
    - **Interoperable:** a portable data format
    - Example: U.S. Health records
    - Transfer is hard, even for the *same* patient

DRAFT

Existing Solutions

DRAFT

[1]Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2]Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3]Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — No logic engine. Hacky tooling/scripts
- **Cyc:**[3]
  - ▸ Largest **knowledge base** to date
  - \+ Sophisticated logic engine
  - — Hard-coded, proprietary rules

DRAFT

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Proposed Solution

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]
  - Represents *expansive* nature of language

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Goals

- **Universality:** be completely expressive

DRAFT

Goals

- **Universality:** be completely expressive (**done**)

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**In progress**)

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**In progress**)
  - Be efficient

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**In progress**)
  - Be efficient
  - Can add optimizations *in* Welkin

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**In progress**)
  - Be efficient
  - Can add optimizations *in* Welkin
- **Standardization:** be completely *unambiguous*

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**In progress**)
  - Be efficient
  - Can add optimizations *in* Welkin
- **Standardization:** be completely *unambiguous* (**In progress**)

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**In progress**)
  - Be efficient
  - Can add optimizations *in* Welkin
- **Standardization:** be completely *unambiguous* (**In progress**)
  - Have a formal specification

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

General Requirements

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined meaning

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined meaning
   - Avoid meaningless symbols!

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined meaning
   - Avoid meaningless symbols!
   - Symbol Grounding Problem[2]

DRAFT

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined meaning
   - Avoid meaningless symbols!
   - Symbol Grounding Problem[2]
   - Treat as free parameter in theory

- Achieve both via **representations**

DRAFT

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

Representations

DRAFT

Representations

- Centered around *representing* things

DRAFT

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c

DRAFT

Representations

- Centered around *representing* things
- $a \mathrel{—} c \to b$: a **represents** b **in context** c
  - a: **sign**

DRAFT

Representations

- Centered around *representing* things
- $a \mathrel{-\!\!-} c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**

DRAFT

Representations

- Centered around *representing* things
- $a \mathrel{—} c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**
  - c: **context**

DRAFT

Representations

- Centered around *representing* things
- $a \mathrel{-\!\!-} c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**
  - c: **context**
- Track *faithfulness* of representations via **truth**

DRAFT

Units

DRAFT

Units

- Units are **finite programs**
- Built from **Handles**

DRAFT

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)

DRAFT

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning

DRAFT

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!

DRAFT

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!
- Each unit contains its own **context** of representations
  - Written with braces $\{a, b, \ldots, z\}$

DRAFT

Units

- Units are **finite programs**
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!
- Each unit contains its own **context** of representations
  - Written with braces $\{a, b, \ldots, z\}$
  - Representations are *also* units

DRAFT

Information

DRAFT

Information

- Representations = **Truth**

DRAFT

Information

- Representations = **Truth**
- Information = **Proofs**

DRAFT

Information

- Representations = **Truth**
- Information = **Proofs**
  - Includes simple facts

DRAFT

Information

- Representations = **Truth**
- Information = **Proofs**
  - Includes simple facts
  - Expands to complex truth systems

DRAFT

Information

- Representations = **Truth**
- Information = **Proofs**
  - Includes simple facts
  - Expands to complex truth systems
- Major result: can express *any* "reliable" proof

DRAFT

Information

- Representations = **Truth**
- Information = **Proofs**
  - Includes simple facts
  - Expands to complex truth systems
- Major result: can express *any* "reliable" proof
  - More details later! (Metatheory)

DRAFT

Rules

- Three categories:

DRAFT

Rules

- Three categories:
  - **Primary:**

DRAFT

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context

DRAFT

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)

DRAFT

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
  - **Empty**

DRAFT

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
  - **Empty**
    - Defines empty unit

DRAFT

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
  - **Empty**
    - Defines empty unit
    - Enables exclusions

DRAFT

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
  - **Empty**
    - Defines empty unit
    - Enables exclusions
  - **Lattice**

DRAFT

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
  - **Empty**
    - Defines empty unit
    - Enables exclusions
  - **Lattice**
    - Makes units position independent

DRAFT

Rules

- Three categories:
  - **Primary:**
    - Transitivity in a context
    - Congruence (build new units)
  - **Empty**
    - Defines empty unit
    - Enables exclusions
  - **Lattice**
    - Makes units position independent
- For details: consult thesis

DRAFT

Turing Completeness

DRAFT

Turing Completeness

- Part of Universality

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - For details: consult thesis
- Later:

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - For details: consult thesis
- Later: completeness for **proofs**!

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

Syntax

- Two main parts

DRAFT

Syntax

- Two main parts
  - Encoding: characters

DRAFT

Syntax

- Two main parts
  - Encoding: characters
    - Building blocks of strings

DRAFT

Syntax

- Two main parts
  - Encoding: characters
    - Building blocks of strings
  - Grammar: strings allowed by language
    - Punctuation
    - How strings are combined

DRAFT

Encoding

- Characters written in `type-writer font`[1]
- Define US-ASCII: Printable (codes: 32-126) + `EOF` (code: 127)
- Character classes: refer to thesis

DRAFT

[1]Font used: Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1 (https://github.com/intel/intel-one-mono/blob/main/OFL.txt).

US-ASCII

Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph
32	20	Space	46	2E	.	60	3C	<	74	4A	J	88	58	X	102	66	f	116	74	t
33	21	!	47	2F	/	61	3D	=	75	4B	K	89	59	Y	103	67	g	117	75	u
34	22	"	48	30	0	62	3E	>	76	4C	L	90	5A	Z	104	68	h	118	76	v
35	23	#	49	31	1	63	3F	?	77	4D	M	91	5B	[	105	69	i	119	77	w
36	24	$	50	32	2	64	40	@	78	4E	N	92	5C	\	106	6A	j	120	78	x
37	25	%	51	33	3	65	41	A	79	4F	O	93	5D	]	107	6B	k	121	79	y
38	26	&	52	34	4	66	42	B	80	50	P	94	5E	^	108	6C	l	122	7A	z
39	27	'	53	35	5	67	43	C	81	51	Q	95	5F	_	109	6D	m	123	7B	{
40	28	(	54	36	6	68	44	D	82	52	R	96	60	`	110	6E	n	124	7C	|
41	29	)	55	37	7	69	45	E	83	53	S	97	61	a	111	6F	o	125	7D	}
42	2A	*	56	38	8	70	46	F	84	54	T	98	62	b	112	70	p	126	7E	~
43	2B	+	57	39	9	71	47	G	85	55	U	99	63	c	113	71	q			
44	2C	,	58	3A	:	72	48	H	86	56	V	100	64	d	114	72	r			
45	2D	-	59	3B	;	73	49	I	87	57	W	101	65	e	115	73	s			

Table 1: Printable US-ASCII codes and glyphs.

DRAFT

Grammar

- Build on two papers (details in thesis)
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity
    - Also ensures efficiency
  - **Notes**:

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

DRAFT

Grammar

- Build on two papers (details in thesis)
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity
    - Also ensures efficiency
  - **Notes**:
    - Current proofs are done

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

DRAFT

Grammar

- Build on two papers (details in thesis)
  - First: **Invertible Syntax Descriptions**[1]
    - Processes strings, forwards and back
  - Second: Edelmann et. al.[2]
    - Used in proof of unambiguity
    - Also ensures efficiency
  - **Notes**:
    - Current proofs are done
    - Grammar is **not** finalized

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

```
start := units - lexeme -> EOF,
units := "" | {unit - lexeme -> *{"",
"," - lexeme -> units}},
unit := node - lexeme -> *{
  ":=" - lexeme -> choices,
  *{"", arc - lexeme -> chain}
},
choices := *{"", "|"} - lexeme ->
choices_list,
choices_list := *{"",
  chain - lexeme -> *{"", "|" - lexeme
-> choice_list}
},
chain := *{"", "*"} - seq -> path -
lexeme -> *{"", arc - lexeme ->
chain},
arc := right_arc | other_arc,
right_arc := "-" - seq -> node - seq -
> "->",
other_arc := "<-" - seq -> node - seq
-> *{"-", "->"},
node := *{"", "*"} - seq -> path,
path := *{"", modifiers} - seq ->
trailer - seq -> *{"", graph},
modifiers := *{
  "@",
  "~" - seq -> *{"", "@"}
},
trailer := segment - seq -> *{"", "."
- seq -> trailer},
segment := HANDLE - lexeme -> *{"",
graph},
graph :=  "{" - lexeme -> units -
lexeme -> "}",
HANDLE := ID | STRING,
STRING := SQ_STRING | DQ_STRING,
SQ_STRING := "'" - seq -> *{SQ_CHAR,
"\'"} - seq_many_till -> "'",
DQ_STRING := '"' - seq -> *{DQ_CHAR,
'\"'} - seq_many_till -> '"',
ID := ID_CHAR - seq_many_till ->
*{RESERVED, WHITESPACE}
```

Listing 3: Full Welkin grammar.

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

Formal System Limits

- Many theorems on limit:s

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.
- Usual approach: *chain of stronger theories*

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)
  - Technically: this is enough, WITH non-constructive methods[1]

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)
  - Technically: this is enough, WITH non-constructive methods[1]
  - Problem: want to identify + trust proofs!

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Artëmov's Selector Proofs

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T show each T-proof p is consistent (no $\perp$)

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T show each T-proof p is consistent (no $\perp$)
- This works...

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T show each T-proof p is consistent (no $\perp$)
- This works... *almost*

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T show each T-proof p is consistent (no $\perp$)
- This works... *almost*
  - If T inconsistent: arises from *some* finite counter-example

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T show each T-proof p is consistent (no $\perp$)
- This works... *almost*
  - If T inconsistent: arises from *some* finite counter-example
  - Proof above: meant to show *no* such example exists!

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T show each T-proof p is consistent (no $\perp$)
- This works... *almost*
  - If T inconsistent: arises from *some* finite counter-example
  - Proof above: meant to show *no* such example exists!
  - Problem: could use $\perp$ to prove!

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm[1]
  - Proves a weaker property: **selector-consistency**
- Approach: over a theory T, build a computable function *over* T-proofs
  1. Enumerate through all T-proofs in a meta-theory (e.g., PA)
  2. Using T show each T-proof p is consistent (no $\perp$)
- This works... *almost*
  - If T inconsistent: arises from *some* finite counter-example
  - Proof above: meant to show *no* such example exists!
  - Problem: could use $\perp$ to prove!
  - Need to add restriction: **must be constructive**

DRAFT

[1] Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

DRAFT

[1]Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:

DRAFT

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth

DRAFT

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth
  - *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$

DRAFT

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth
  - *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$
    - Explosion: $\bot \Rightarrow \varphi$

DRAFT

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique![1]
- Let T be a first-order, RE theory
- T is **serial-sound**, if for some total function s:
  - Accepts all proofs + preserves truth
  - *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$
    - Explosion: $\bot \Rightarrow \varphi$
- T is **self-verifying** if, *in* PA, T can prove its own serial-soundness
  - Notation: $\mathrm{PA} \vdash_T \varphi$ if $T \vdash \varphi$ and T self-verifying

DRAFT

[1] Consult: Artemov (2024). Serial properties, selector proofs and the provability of consistency. J. Logic Comput. 35(3).

Reliable Proofs

- How to generalize to *any* theory?

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$
    - Idea: check for sound *restricted* theory
- Let T_1 be an RE theory, φ a sentence

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$
    - Idea: check for sound *restricted* theory
- Let T_1 be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \supseteq T_1$, $S \neq \varnothing$

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$
    - Idea: check for sound *restricted* theory
- Let T_1 be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \supseteq T_1$, $S \neq \varnothing$
  - p_S: constructs some self-verifying $S' \supseteq S$

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$
    - Idea: check for sound *restricted* theory
- Let T_1 be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \supseteq T_1$, $S \neq \varnothing$
  - p_S: constructs some self-verifying $S' \supseteq S$
  - p_φ: S-proof of φ

DRAFT

Proof Completeness

- **Note:** still has incompleteness!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- How to reach limit on proofs?

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- How to reach limit on proofs?
  - Basic idea: ZFC is self-verifying!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- How to reach limit on proofs?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- How to reach limit on proofs?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!
  - **Huge** leap in power; how to extend it?

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- How to reach limit on proofs?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!
  - **Huge** leap in power; how to extend it?
  - Answer: comprehension!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - However, a theory *can* be self-verifying!
- How to reach limit on proofs?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!
  - **Huge** leap in power; how to extend it?
  - Answer: comprehension!
  - Details in thesis

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

Conclusion

- Welkin is a universal information language

DRAFT

Conclusion

- Welkin is a universal information language
  - Provided basic rules

DRAFT

Conclusion

- Welkin is a universal information language
  - Provided basic rules
  - Provided syntax

DRAFT

Conclusion

- Welkin is a universal information language
  - Provided basic rules
  - Provided syntax
  - Provided metatheory

DRAFT

Conclusion

- Welkin is a universal information language
  - Provided basic rules
  - Provided syntax
  - Provided metatheory
- Goals:

DRAFT

Conclusion

- Welkin is a universal information language
  - Provided basic rules
  - Provided syntax
  - Provided metatheory
- Goals:
  - Universal (**done**)

DRAFT

Conclusion

- Welkin is a universal information language
  - Provided basic rules
  - Provided syntax
  - Provided metatheory
- Goals:
  - Universal (**done**)
  - Scalable (**in progress**)

DRAFT

Conclusion

- Welkin is a universal information language
  - Provided basic rules
  - Provided syntax
  - Provided metatheory
- Goals:
  - Universal (**done**)
  - Scalable (**in progress**)
  - Standardization (**in progress**)

DRAFT

Comparisons to Existing Solutions

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:
    - Formal definitions and grammar

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:
    - Formal definitions and grammar
    - Logical engine

DRAFT

Comparisons to Existing Solutions (cont')

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:
    - Flexible data format

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:
    - Flexible data format
    - Open specification

DRAFT

Future Work

DRAFT

Future Work

- Write the Standard

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation
- Information Search Programs

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation
- Information Search Programs
- Revision System

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation
- Information Search Programs
- Revision System
- Package Management

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation
- Information Search Programs
- Revision System
- Package Management

DRAFT

Thank you!

Questions?

DRAFT